PSynergy.

An Independent Journal for PlayStation 2 Linux Kit Developers and Enthusiasts.

Issue 00, July 2003.

Editor-in-chief: Patryk Laurent
Editors: Editor One
Advisory Board: Sparky, Sauce.

Table of Contents

Editorial....3
Features....4
Developing for the PS2 Linux Kit RTE (Part 1)....4
Unaligned pointers are a performance hit, not cause of crash....5
Asm speedups and new things about the main CPU....8
Project Spotlight....10
The SPS2 Library Project....10
New Projects/Resources....11
PS2 Native Mode for the RTE....11
Linux framebuffer driver released....11
NCSA PS2 Development....11
Tips....12
POW function and PS2-specific math....12
Why swap bits 3 and 4?....12
Q&A and Feedback....14

Article Submission Guidlines.

You may submit a copy of your article manuscript to patryk@playstation2-linux.com. For your article to even be considered, it must be related to development for the Playstation 2 Linux kit. Please note that this includes programming for the RTE, so long as anyone with the Kit can learn/use it. All submissions will be edited for content, clarity and brevity. A suggested template for articles can be obtained at this URL ____. Finally, by submitting your article, you agree that it will be distributed under the GNU Free Documentation License.

This journal is licensed under the GNU Free Documentation License.
Journal text is in Charter, point size 9. Source code listings are in New Courier.

Editorial

Welcome to our first issue of PSynergy! Our goal is to provide the Playstation 2 Linux Kit developer community with a place for record and reference, and where findings can be published. Submission is open to all, please see the submission guidelines. We would like this journal to help generate interest in the kit for

- current Playstation 2 Linux Kit developers,
- knowledgeable developers who would otherwise ignore the Playstation2, and
- potential new developers.

We plan to release a new issue every month. In the case of a high volume of quality content submitted to us there will be Special issues of PSynergy released on an irregular basis.

I have been greatly impressed and satisfied with the linux kit in every respect, I only find fault with the very limited amount of time and energy I personally have had to invest into developing for the kit. Keeping up to date with all the neat stuff people are doing with it has proved to be challenging. It is my hope that as PSynergy chronicles the community's way through learning about the PS2 and way to apply it, PSynergy can enrichen the development and exploration experience for us all.

Welcome!

Patryk Laurent
Editor-in-Chief, PSynergy Journal
patryk@playstation2-linux.com

Features

Feature: **Developing for the PS2 Linux Kit RTE (Part 1)**

Marcus R. Brown
[text not yet provided; please continue]

***Feature*: Unaligned pointers are a performance hit, not cause of crash**
http://playstation2-linux.com/forum/message.php?msg_id=30644

Steven Osman (*sauce)*

I think the following information is good for all PS2 developers to know, but it will probably be of much more interest to people using **sps2** in attempting to squeeze as much performance out of their PlayStation2. What is discussed here doesn't apply to RTE programmers because what is being discussed here is specific in overcoming the restrictions (read: protection) imposed by the Linux kernel.

I'd like to thank mrbrown, Sparky and ldesnogu for some interesting insight and conversation regarding this issue.

Those of you who have programmed directly to the PS2 hardware or other platforms such as the PSX are accustomed to those extremely annoying crashes that are caused when data at an unaligned pointer is loaded. For instance, if you issue a 4-byte load instruction (i.e. **lw**) and the pointer you use is not word (4 byte) aligned, then your application would normally crash.

Some of you (non-Linux heads) may be surprised to find out that this does not crash on your system. The reason for this is that the linux kernel actually traps the address alignment exception and emulates an aligned load even though your load just broke. For those of you concerned about highly efficient code, this should raise major red flags in terms of cache usage, exceptions, context switch pentalties, as well as the overhead of the emulation code.

There are two bits of good news.

First, if you're a glutton for punishment (or want to make sure that your code executes really quickly), you can disable this emulation. It will cause your application to crash with a Bus Error when you do cause a misaligned load/store, but at least you won't be suffering very poor performance with no indication as to why that is happening (emulation doesn't display any messages whatsoever).

You must include <sys/sysmips.h> and then issue:

```
sysmips(MIPS_FIXADE, 0, 0, 0);
```

to disable the emulation. If you want to re-enable emulation, issue:

```
sysmips(MIPS_FIXADE, 1, 0, 0);
```

Second, should you choose to disable it or not, there's a really easy fix that I stuck in my kernel source that informs me when such an unaligned access happens. Basically, you need to edit your arch/mips/unaligned.c in your kernel source tree and in the do_ade() and add (*see Snippet 1.1*):

```
printk("Unaligned memory access at %p, address %p, "
            "PID %i (%s)\n",
            (void *) pc, (void *) regs->cp0_badvaddr, (int) current->pid,
            current->comm);
```
Snippet 1.1

between the following two if statements (*see Snippet 1.2*):

```
if (compute_return_epc(regs))
            return;
if ((current->tss.mflags & MF_FIXADE) == 0)
            goto sigbus;
```
Snippet 1.2

(i.e. right after the "return;")

Of course, if you have 5 billion unaligned accesses, your code will become EVEN SLOWER now because it'll be dumping the above debugging data which includes the addess of the instruction, the memory address, and the pid/process name in error. At least you'll now know it's happening and can fix it. Don't be surprised if many of your favorite applications are doing this often and, as such, suffering poor performance. There's a good chance that these applications weren't originally designed with the MIPS alignment issues in mind.

For those of you curious to see the emulation in

action, consider the code I'm attaching below. It takes an array of two ints which are 4-byte aligned, but makes a pointer that is two bytes into the first int (so it's no longer 4-byte aligned). There's a piece of assembly code that does an lw/sw on this pointer and succeeds despite the alignment problem. The code then disables the alignment emulation and reruns the same assembly code which causes the process to crash.

If you don't feel like running it, the program output is:

```
A: Ints are 0:00000000 1:00000000
B: Ints are 0:00010000 1:00000000
Bus error
```

And if you've put the debugging statement in your kernel source you'll see something like:

```
Jan 26 01:49:28 emotion kernel: Unaligned memory access at 00400c00, address 7ffffaf2,
PID 1208 (unaligned)
Jan 26 01:49:28 emotion kernel: Unaligned memory access at 00400c08, address 7ffffaf2,
PID 1208 (unaligned)
```

Well, enjoy. mrbrown and I are looking into "nicer" ways of working with this, we'll keep you posted. But in the meantime, these two little tricks will definitely help performance optimizations.

See Listing 1.1 below for an illustration of the contents of this article.

```
// This program deliberately performs unaligned loads and stores
// The first time the process is rescued by the linux kernel, but
// once that rescue is disabled, the process crashes the second time
// around
#include <stdlib.h>
#include <stdio.h>
#include <sys/sysmips.h>

int iTemp;

void main() {
// Two words, will be 4-byte aligned by the compiler
int aiWords[2]={0, 0};
// Shorts are only aligned at 2 bytes
short *psShort;


psShort=(short *) aiWords;
// No longer 4 byte aligned after this
psShort++;

// Expect 0 and 0
printf("A: Ints are 0:%08x 1:%08x\n", aiWords[0], aiWords[1]);

// Here's asm that takes in a pointer we know is not 4-byte aligned
// and tries to load a word. Because of emulation, it will work.
asm __volatile__("
lw %0, 0x00(%1);
addi %0, 1
sw %0, 0x00(%1)
" : "=r" (iTemp) : "r" (psShort));

// Expect 0x10000 and 0
printf("B: Ints are 0:%08x 1:%08x\n", aiWords[0], aiWords[1]);

// Disable emulation
sysmips(MIPS_FIXADE, 0, 0, 0);

// Now the same unaligned access will result in a bus error.
asm __volatile__("
lw %0, 0x00(%1);
addi %0, 1
sw %0, 0x00(%1)
" : "=r" (iTemp) : "r" (psShort));

// If this worked, we'd expect 0x20000 and 0
printf("C: Ints are 0:%08x 1:%08x\n", aiWords[0], aiWords[1]);

return 0;
}
```

Listing 1.1

Feature: **Asm speedups and new things about the main CPU**
http://playstation2-linux.com/forum/message.php?msg_id=30646

Morten Mikkelsen (*sparky)*

Dunch's post has inspired me to supply you with a little bit of information on speeding up code for the EECore -- the main CPU on the PlayStation2. In many ways, optimizing code for the EECore is actually more difficult than speeding up VU code but it's also very interesting and educational.

In various places of the manual you'll see something refered to as throughput/latency. *Throughput* is how soon you can do an instruction of the same type as the one you just did but one that doesn't use the result of your current instruction and *latency* is how soon you can issue an instruction using/reading the result of the instruction.

For instance most of the FPU instructions are 1/4 instructions meaning throughput is 1 and latency is 4.

An example of this *(see Snippet 2.1):*

```
mul.s $f3, $f0, $f0       # 1. cycle
add.s $f4, $f0, $f1       # 2. cycle
sub.s $f5, $f1, $f2       # 3. cycle
abs.s $f6, $f0            # 4. cycle
mul.s $f7, $f3, $f0       # 5. cycle
```

Snippet 2.1

So the result of $f3 launched at the 1. cycle is ready for reading in the 5. cycle, if you try to read it sooner, the cpu will stall until the result is ready so you might aswell find something usefull to do with the gap inbetween.

The accumulator instructions are nice because there's no penalty adding to the intermediate result which is always in the **acc** register.

```
mula.s $f0, $f0  # 1. cycle, acc = f0*f0
madda.s $f1, $f1 # 2. cycle, acc += f1*f1
madda.s $f2, $f2 # 3. cycle, acc += f2*f2
...
madd.s %0, $fn, $fn # n. cycle,
%0=acc+fn*fn
abs.s tmp, tmp   # n+1. cycle
abs.s tmp2, tmp2 # n+2. cycle
abs.s tmp3, tmp3 # n+3. cycle
add.s %1, %0, %0 # n+4. cycle (%0, ready)
```

Snippet 2.2

And %0 is ready for use 4 cycles after the final madd.s instruction. Again you can just try and use it right away, but the cpu will just stall. By the way GCC for ps2/PS2Linux doesn't know how to use the accumulator instructions.

Integer operations are 1/1 instructions except for the mul and div instructions. Be careful with these since there are macros expanding what you might think is one instruction into several. Such as "**li $t0, 0xdeadbeef**" which expands into "**lui $t0, 0xdead**" and ori "**ori $t0, $t0, 0xbeef**". There's also the "**rol**" which will expand into two shifts and an or, and many many other pseudo instructions.
Using -S will not give you the final assembler code, so if you wish to know for sure if you got the code you were expecting you should use some kind of disassembler, perhaps using **gdb** or **objdump** -- disassemble mycode.elf

I also suggest you insert **.set noreorder** at the top of your loop and **.set reorder** at the bottom -- basicly telling the compiler to butt out. It often inserts more into your code then you bargened for.

Now another interesting aspect is instruction pairing. It's possible to execute several of the instructions '*two in one cycle*' if you setup your code with care. You should have a look at the *EE Core User's Manual* and look for the table called "*Categories of instructions and Routing to Physical pipes*". I'm looking at an older version of Manual just now but I think someone told me it's on page 21, anyway it's easy enough to find it.

2 times * (star) means instruction will need both pipes, 2 times O means it can use either of the two pipes but one of them must be available. A single O means must execute in this pipe.
Now the pairing strategy is simple, couple instructions that don't collide on pipes.

For instance you can do two integer operations in one cycle but you can NOT do two FPU instructions. However you can do one FPU instruction and have it coupled with integer, load/store, branch and a few others.

MM instructions can't couple with anything using either of the integer pipes (i0 and i1) but it can be coupled with a load/store/cop1 move/cop2 move/branch.

If you play around with this I do suggest setting **.set noreorder** at the top and **.set reorder** at the

bottom and double checking your code using **dumpobj** since it's extremely important that you get the instruction sequence you were expecting.

I would also like to add that you can't couple two integer operations if the first is doing a result that the next is reading.

This example executes in two cycles (no pairing):

```
addiu $t0, $0, 7
add $t1, $t0, $t0

Snippet 2.3
```

This example executes as a pair in one cycle.

```
addiu $t0, $0, 7
addiu $t1, $0, 7+7

Snippet 2.4
```

Another thing you might want to know is that loading from memory is same speed as the scratchpad if it's a hit, it's a 1/2 instruction meaning you can do another bus access in the next cycle but you should not use the register until the next cycle after that.

Example:

```
a1: lw $t0, 0(%0) # 1. cycle
a2: nop
b1: lw $t1, 4(%0) # 2. cycle
b2: nop
c1: add $t2, $t0, $t0 # 3. cycle
c2: nop

Snippet 2.5
```

So $t0 is ready for use in the 3. cycle assuming it's a hit or fetched from scratchpad.

If you get a miss the stall will be longer then 30 cycles (use DMA for intensive data reading).

The scratchpad is devided into 4 banks each 4 kb, if you access from a different bank then last read/store you did, then there will be a penalty of 1 cycle and you will not be able to fill the stall with another instruction. You will be stalled unconditionally for 1 cycle. So don't switch banks too often. Another weird thing, you also get a 1 cycle loss when switching sign of offset regardless of the base register. I'll assume in this sample %0 and %1, points to somewhere on the scratchpad.

```
a1: lw $t0, 4(%0) # 1. cycle
a2: nop
b1: lw $t1, -4(%1) # 2. cycle
b2: nop
c1: nop # 4. cuz of stall
c2: nop

Snippet 2.6
```

Again we get an unconditional stall, no chance to fill the gap with other code.

Make sure you fiddle with the base pointers before entering a loop so all your baseregisters (%0, %1 in this case) will provide you with either negative offsets all the way through the loop or possitive all the way.

This is ONLY! an issue when dealing with the scratchpad.

If you think any of this was too strange to understand or badly described don't hesitate to ask.

Project Spotlight

Project Spotlight: Steven “Sauce” Osman's SPS2 Library Project
http://playstation2-linux.com/projects/sps2

Patryk Laurent

1. Introduction

The Playstation2 Linux Kit Version 1.0 first became available in the U.S. during summer 2002. A few months later in November 2002, the SPS2 library was first released to the general public by Steven “Sauce” Osman. SPS2's release was just in time to quell the swarms of questions and pleas for help that were appearing on the Developers forums about various low-level tasks on the PS2 such as performing DMA transfers. Most recently, SPS2 was mentioned in Sony's presentation to the GDC in Australia.
The purpose of this article is to address two questions: Who is SPS2 for and what can the SPS2 library do for you?

2. SPS2 Capabilities

If you are interesting in doing programming for the Playstation2 under Linux that takes advantage of the PS2-specific hardware such as the DMA Controller and the Vector Processing Units (VPUs), you *could* use libps2dev. **libps2dev**, shipped with the Linux Kit under GPL, provides some useful functionality, especially for accessing the VPUs but provides very limited access to the DMA Controller. Instead of hiding the memory locations like libps2dev, **SPS2** provides the developer with direct access to all memory locations involved with PS2 programming.

Sauce's SPS2 derives its power by combining several important feautures. The most important feature of SPS2 is that it allows your application to *directly* access low-level PlayStation2 functions, without requiring your application to make the expensive switch to kernel mode. (There are a couple of exceptions to this, such as when waiting for a dma transfer to complete.) SPS2 thus provides freedom from the programming safeties and restrictions imposed by every Linux kernel. This is achieved having SPS2 be a kernel module[1]. As a kernel module, SPS2 has direct access to the memory locations which are central to developing on the PS2. (The memory locations are documented in the beginning of the EE User Manual). Secondly, SPS2 provides mechanisms for safely allocating and caching aligned blocks of memory. Finally, the library allows you to access the various memory locations in Playstation2 RAM through a standard naming convention. Rumor has it that programming under SPS2 even bears a little resemblance to programming on a T10k!

3. What's in the bundle

At the time of writing of this article, the public release of SPS2 is in version 0.2a. You can download two separate packages: **sps2mod** and **sps2dev**. For end-users of applications developed with SPS2, sps2mod is all that's needed. It contains the source code for the module only, which is easily set up with a one-time make depend, make and make install. The sps2dev package contains extra files, specifically the include files for development and a series of fantastic demos including one that renders 34.2 million textured vertices per second while using no more than 1% of EECore CPU time (see the samples readme.txt file).

Version 0.2a also adds a couple of nice, basic tutorial examples demonstrating how to use SPS2 and some higher level functions in the included *sps2util* for initializing the Graphics Synthesizer (GS). Thus developers can 'hit the ground running' with SPS2 and begin developing right away.

4. The License

Being a budding Playstation2 Linux Kit developer myself, and having struggled with DMA transfers, I was extremely excited to hear the news of SPS2 being released. However, the excitement of several

1 This is especially nice in that you don't have to recompile any kernels.

including myself was checked by the non-GPL (and non-LGPL) license. Key points about the license are that you may not redistribute SPS2, nor are you permitted to distribute any altered forms of the module.

This license reflects Sauce's well-expressed disdain for the GPL-style Open Source. Unlike others who might support GPL because it appears to benefit each member of the community at large, many believe that GPL only makes sense if the developer can benefit from licensing it this way. As far as a library such as SPS2 is concerned, allowing redistribution and modification could only be a disservice in that it increases the headaches of version control, ensuring compatability, and troubleshooting. Making SPS2 GPL could destroy the very essence of the library.

I had an opportunity to have dinner with Sauce several weeks ago and found that he is extremely open to discussions about the pros and cons of licensing. Contact him if you have any questions or qualms about his license at all.

5. Conclusion: The Future of SPS2 and Playstation2 Linux Kit Development

SPS2 is an excellent library that allows one to program directly on the Playstation2 hardware. However, for some this may be too fine a level of detail -- SPS2 is not for the high-level programmer. (See however the latest release -- sps2dev-0.2.0a -- which has some higher level screen handling routines.) An interesting possibility would be for developers to use SPS2 to develop higher-level libraries for specific tasks, thereby increasing SPS2's user base. For example, since SPS2 is a low-level library, it provides access to the control registers for accessing vector units and to the VU memories, but it does not provide high level commands such as vpu1_start(), vpu1_upload_microcode().

No doubt SPS2 will be the basis of many forthcoming games developed under PS2Linux. Will programmers be willing to create and distribute high-level libraries based on SPS2? It will also be interesting to see what kinds of licenses they choose to embrace.

New Projects/Resources

PS2 Native Mode for the RTE
(contact mrbrown)
http://playstation2-linux.com/forum/message.php?msg_id=30411

For those of you interested in running those PS2 demos,

"It reboots the original PS2 operating system from the Linux Kit's RTE, enabling you to run and develop all the fun stuff you can use with Naplink.
Some demos may not work, I presume this is because they don't initialize themselves properly, and the native OS is finnicky about that type of stuff (I'm referring to demos that work under Naplink). But this is not the fault of this program ... I'll writeup something about that later. "

A description of a bug preventing certain ELFs from being booted properly from the RTE is mentioned by mrbrown here
http://playstation2-linux.com/forum/message.php?msg_id=30198

More information on this bug and how it can be fixed can be found in the document below:
https://playstation2-linux.com/docs/haven/display_doc.php?docid=55

Linux framebuffer driver released
(contact alvarov)
http://playstation2-linux.com/forum/message.php?msg_id=31140

Ps2Linux now supports a framebuffer (http://www.linux.org/docs/ldp/ howto/Framebuffer-HOWTO.html) thanks to ps2linux developer alvarov who says, "*This replaces the standard GS driver that comes with the kit with a standard framebuffer interface, thus enabling many console-based applications to execute on the PS2. Getting rid of X windows in a 32Mb-RAM machine sounds like a good idea...*"

Several good questions were raised in the form. The project page is here:
http://playstation2-linux.com/projects/ps2gsfb

NCSA PS2 Development
(contact gevmage)
http://playstation2-linux.com/forum/message.php?msg_id=29301

Work on a linear algebra library and other interesting projects is ongoing at
http://arrakis.ncsa.uiuc.edu/ps2/index.php

Tips

POW function and PS2-specific math
(contact istephen and sparky)
http://playstation2-linux.com/forum/message.php?msg_id=28902
http://playstation2-linux.com/forum/message.php?msg_id=29879

istephen and sparky had an interesting discussion on optimizing the POW function (Taylor polynomials) on the PS2. You'll find EECore code by following the links above.

Why swap bits 3 and 4?
(contact jase and bbadillo)

```
unsigned int swapBits3and4(unsigned int value) // works on 32bit value
{
    return (value&0xe7e7e7e7) | ((value&0x08080808)<<1)|((value&amp;0x10101010)>>1);
}
```

Palettes on the PS2 are stored just like textures - which means that they're "swizzled" or stored in a slightly rearranged format. For a 32bit palette, this swizzling manifests itself as swapping over the 2nd and 3rd block of 8 colours out of every 32 colours in the palette. This can also be acheived by mangling the index in the way shown in this code.

Q&A and Feedback

As this is a first issue we do not have feedback from the public at large, but we do have feedback from people who have provided intial reviews of this first issue.